

Operators in Python

1. Assignment Operators

Assignment operators are used to assign values to variables. The simplest one is = which assigns the value on the right to the variable on the left. There are also compound assignment operators that combine arithmetic operations with assignment.

Common Assignment Operators:

- =: Assigns value on the right to the variable on the left.
- +=: Adds right operand to the left operand and assigns the result to the left operand.
- -=: Subtracts the right operand from the left operand and assigns the result to the left operand.
- *=: Multiplies the left operand by the right operand and assigns the result to the left operand.
- /=: Divides the left operand by the right operand and assigns the result to the left operand.
- %=: Takes modulus of left operand by right operand and assigns the result to the left operand.

Examples:

```
x = 5      # Assigns 5 to x
x += 3     # Equivalent to x = x + 3, now x is 8
x -= 2     # Equivalent to x = x - 2, now x is 6
x *= 4     # Equivalent to x = x * 4, now x is 24
x /= 6     # Equivalent to x = x / 6, now x is 4.0
```

2. Comparison Operators

Comparison operators are used to compare two values. They return either True or False depending on the condition.

Common Comparison Operators:

- ==: Checks if two values are equal.
- !=: Checks if two values are not equal.
- >: Checks if the left operand is greater than the right operand.
- <: Checks if the left operand is less than the right operand.
- >=: Checks if the left operand is greater than or equal to the right operand.
- <=: Checks if the left operand is less than or equal to the right operand.

Examples:

a = 10

b = 20

print(a == b) # Output: False

print(a != b) # Output: True

print(a > b) # Output: False

print(a < b) # Output: True

print(a >= 10) # Output: True

print(b <= 25) # Output: True

3. Logical Operators

Logical operators are used to combine conditional statements. They evaluate expressions and return either True or False.

Common Logical Operators:

- and: Returns True if both conditions are true.
- or: Returns True if at least one condition is true.
- not: Reverses the logical state of its operand (True becomes False, and vice versa).

Examples:

x = 5

y = 10

z = 15

and operator

print(x > 0 and y > 5) # Output: True (both conditions are True)

or operator

print(x > 10 or z > 10) # Output: True (one of the conditions is True)

not operator

print(not(x > 10)) # Output: True (reverses False to True)

4. Membership Operators

Membership operators test for membership within a sequence, such as a list, string, or tuple. They return True or False based on whether the value is found in the sequence.

Membership Operators:

- `in`: Returns True if the specified value is found in the sequence.
- `not in`: Returns True if the specified value is not found in the sequence.

Examples:

```
my_list = [1, 2, 3, 4, 5]
```

```
my_string = "Python"
```

```
print(3 in my_list)          # Output: True (3 is in the list)
```

```
print(6 not in my_list)      # Output: True (6 is not in the list)
```

```
print("P" in my_string)      # Output: True ("P" is in the string)
```

```
print("z" not in my_string)  # Output: True ("z" is not in the string)
```

5. Bitwise Operators

Bitwise operators perform operations on binary representations of integers. These operators are useful for low-level programming tasks like working with bits and bytes.

Common Bitwise Operators:

- `&`: Bitwise AND (sets each bit to 1 if both bits are 1).
- `|`: Bitwise OR (sets each bit to 1 if one of the bits is 1).
- `^`: Bitwise XOR (sets each bit to 1 if only one of the bits is 1).
- `~`: Bitwise NOT (inverts all the bits).
- `<<`: Left shift (shifts bits to the left by a specified number of positions).
- `>>`: Right shift (shifts bits to the right by a specified number of positions).

Examples:

```
a = 5 # In binary: 101
```

```
b = 3 # In binary: 011
```

```
# Bitwise AND
```

```
print(a & b)          # Output: 1 (binary: 001)
```

```
# Bitwise OR
```

```
print(a | b)          # Output: 7 (binary: 111)
```

```
# Bitwise XOR
print(a ^ b)          # Output: 6 (binary: 110)

# Bitwise NOT
print(~a)             # Output: -6 (inverts all bits)

# Left shift
print(a << 1)         # Output: 10 (binary: 1010)

# Right shift
print(a >> 1)         # Output: 2 (binary: 010)
```

6. Arithmetic Operators

Python supports basic arithmetic operations like addition, subtraction, multiplication, division, and more.

Common Operators:

- + (Addition)
- - (Subtraction)
- * (Multiplication)
- / (Division)
- // (Floor Division)
- % (Modulus)
- ** (Exponentiation)

Examples:

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```